

TECHNICAL DOCUMENTATION

ASAR Sanctuary

Comprehensive technical reference for the ASAR Engine (FastAPI) and Sanctuary front-end (React SPA). Covers architecture, REST APIs, authentication, persistence, and configuration.

FastAPI Backend | React SPA | Quran Integrations

APRIL 2026

Table of Contents

1. Repository Layout	3
2. High-Level Architecture	3
3. Authentication and Identity	5
3.1 Demo Login	5
3.2 Quran Foundation OAuth2 + PKCE	5
3.3 JWT Usage	5
3.4 Auth API Summary	5
3.5 UserMe Contract (Profile JSON)	6
3.6 Email/Password Login	7
4. Session ID Semantics	7
5. Versioned REST API (/api/v1)	8
5.1 Chat	8
5.2 Streak and Habit Ledger	8
5.3 Quran Content (Proxied)	9
5.4 Chat History	9
5.5 Bookmarks (Authenticated Only)	10
6. Two Chat Pipelines	10
6.1 POST /chat - Verified Assistant	11
6.2 POST /chat/message - REST RAG	11
7. Persistence (SQLite)	11
8. Configuration Reference	12
8.1 Core and Security	12
8.2 LongCat (LLM)	13
8.3 Quran.com Content API	13
8.4 REST RAG Tuning	14

8.5 Quran Foundation OAuth and APIs	14
8.6 Ledger	15
8.7 Front-end Environment	15
9. Front-End: Routing and Guards	15
10. Front-End: Pages and Data Sources	16
11. Testing	17
12. Operational Notes (12-Factor)	18
12.1 Production Checklist	18
12.2 OAuth Redirect URI Troubleshooting	18
12.3 General Operational Guidelines	19

1. Repository Layout

The ASAR Sanctuary repository is organized into two primary top-level directories: **Back-End** and **Front-End**. The backend follows a layered architecture pattern with clear separation between API routes, service modules, data models, and core configuration. The frontend follows a feature-based organization with pages, contexts, layouts, and shared utility libraries. This structure ensures maintainability and clear ownership boundaries between the FastAPI engine and the React SPA.

Path	Role
Back-End/app/main.py	FastAPI application entry, CORS, router mounting, async lifespan (shared HTTP client + OpenAI-compatible client).
Back-End/app/api/routes/	Versioned API under /api/v1: composed routers (chat, streak, quran_proxy, history_routes, bookmarks).
Back-End/app/api/auth/	Auth endpoints: demo JWT, Quran Foundation OAuth, /me profile and onboarding (__init__.py composes routers).
Back-End/app/api/deps.py	JWT dependencies: get_current_user, get_current_user_optional.
Back-End/app/core/config.py	Pydantic settings from environment variables (12-factor style).
Back-End/app/models/domain.py	SQLAlchemy ORM models for SQLite persistence layer.
Back-End/app/models/schemas.py	Pydantic request/response schemas for API validation.
Back-End/app/services/	Quran HTTP, AI, RAG chat, streak helpers, bookmark sync, user API services.
Front-End/src/App.tsx	React Router routes and authentication guard composition.
Front-End/src/lib/apiClient.ts	Axios instance with X-Session-ID and Authorization: Bearer interceptors.
Front-End/vite.config.ts	Dev proxy: /api forwarded to http://127.0.0.1:8000.

Table 1: Key repository paths and their responsibilities

2. High-Level Architecture

The ASAR Sanctuary platform follows a clean separation of concerns across multiple architectural layers. The browser loads a Vite-built React single-page application (SPA) that communicates with the FastAPI engine through a versioned

REST API under **/api/v1**. During development, API calls from the SPA go through a Vite dev server proxy to avoid CORS friction, while in production the SPA and engine typically share the same origin. The engine handles all API routing, authentication, business logic, and persistence through its layered internal architecture.

The engine talks to **SQLite** for application state, to the **Quran.com HTTP APIs** for Quranic content retrieval and search, optionally to the **Quran Foundation user APIs** for OAuth-linked accounts including activity synchronization and bookmark sync, and to **LongCat** (an OpenAI-compatible HTTPS endpoint) for LLM completions. The engine creates a shared `httpx.AsyncClient` for upstream HTTP calls and an `AsyncOpenAI` client for LLM interactions during its async lifespan, ensuring efficient resource management across all request handlers. The front-end is a **PWA** featuring `@khmyznikov/pwa-install` for cross-platform install prompts on Android, Desktop, and a guided walkthrough for iOS devices.

On startup, the FastAPI engine performs several critical initialization steps: it creates a shared `httpx.AsyncClient` registered with the Quran service module, creates an `AsyncOpenAI` client configured with `LONGCAT_API_KEY` and `LONGCAT_BASE_URL` registered with the AI service module, calls `ensure_demo_account_ready` to guarantee the demo user row exists, and configures CORS using the `CORS_ORIGINS` comma-separated list with `allow_credentials=True` to support OAuth PKCE cookies. This startup sequence ensures all dependencies are ready before the first request arrives.

Layer	Component	Details
Browser	Sanctuary SPA	Vite-built React SPA; calls <code>/api/v1/*</code> endpoints.
Dev Proxy	Vite Dev Server	Forwards <code>/api</code> requests to <code>http://127.0.0.1:8000</code> during development.
Backend	ASAR Engine (FastAPI)	API routing, auth, business logic, SQLite persistence.
External	Quran Content API	Verse data, search, chapters, translations from Quran.com endpoints.
External	Quran Foundation User API	OAuth-linked user activity sync and cloud bookmark sync.
External	LongCat (OpenAI-compat)	LLM completions for verified chat and REST RAG chat pipelines.
Frontend	PWA Install	<code>@khmyznikov/pwa-install</code> for cross-platform native-style install.

Table 2: System architecture layers and components

3. Authentication and Identity

3.1 Demo Login

The Demo Login mechanism provides instant access to the application without requiring external account linkage. The endpoint **POST /api/v1/auth/demo** issues a JWT for the configured demo user when **ENABLE_DEMO_LOGIN** is set to true and **DEMO_USER_PASSWORD** is provided. The front-end calls this endpoint from AuthContext.tsx via loginDemo() on WelcomePage.tsx. This approach is critical for reducing onboarding friction during hackathon demos, initial user trials, and testing scenarios where creating a full OAuth account would be impractical or undesirable.

3.2 Quran Foundation OAuth2 + PKCE

The primary authentication flow uses the OAuth2 Authorization Code grant with PKCE (Proof Key for Code Exchange) for enhanced security. The browser navigates to **GET /api/v1/auth/quran/start** on the engine origin (not the SPA origin). The handler sets an HTTP-only **asar_oauth_pkce** cookie (JWT-encoded verifier and state) and redirects to the Foundation authorize URL. After user consent, **GET /api/v1/auth/callback** exchanges the authorization code, upserts a User record, stores quran_access_token and quran_refresh_token, issues an ASAR JWT, and redirects to FRONTEND_AFTER_OAUTH_URL with the token in a URL fragment (#asar_token=...). The SPA completes sign-in on QuranOAuthLanding.tsx. Critically, the OAuth start must hit the same host as QURAN_OAUTH_REDIRECT_URI so that the PKCE cookie is sent on callback.

3.3 JWT Usage

The system uses HTTPBearer as an optional dependency: a missing or invalid token yields anonymous flow where applicable, while a valid token provides full authenticated access. The JWT sub claim carries the numeric user ID as a string. The decode_access_token function validates both expiry and signature using the configured JWT_SECRET_KEY and JWT_ALGORITHM. This design allows the same endpoints to serve both anonymous exploratory users and authenticated users with full feature access.

3.4 Auth API Summary

Method	Path	Auth	Purpose
--------	------	------	---------

POST	/api/v1/auth/demo	None	Issue a demo JWT for instant feature access.
GET	/api/v1/auth/quran/start	None	Start OAuth2 PKCE flow; sets PKCE cookie and redirects to Foundation.
GET	/api/v1/auth/callback	Cookie+Query	Code exchange, user token storage, redirect with ASAR JWT fragment.
GET	/api/v1/auth/me	Bearer	Full UserMe profile with onboarding, streak, and cursor data.
PATCH	/api/v1/auth/me/ledger-timezone	Bearer	Set user.ledger_timezone (IANA); validated with ZoneInfo.
POST	/api/v1/auth/me/reading-reset	Bearer	Reset sequential reading cursor inside current scope.
PATCH	/api/v1/auth/me/onboarding	Bearer	Complete or update onboarding preferences and reading cursor.
GET	/api/v1/auth/me/recommended-verse	Bearer	Get next recommended verse_key from policy or cursor.

Table 3: Authentication API endpoints

3.5 UserMe Contract (Profile JSON)

The UserMe profile object is built in the `build_user_me` helper function and returned by the GET `/api/v1/auth/me` endpoint. It aggregates data from multiple database tables to provide the front-end with a single comprehensive user state object. The following table documents each field in the UserMe contract and its meaning.

Field	Meaning
asar_session_id	Server-owned UUID for chat, streak, and history when authenticated.
ledger_timezone	Effective IANA zone for mark-complete "day" (ASAR_LEDGER_TIMEZONE or user override).
onboarding_completed	Boolean: true when onboarding_completed_at is not null.
onboarding_*	Goal, level, time budget, journey mode, topic tag captured during onboarding flow.
recommended_verse_key	Next verse suggestion computed by the onboarding policy helper.
current_verse_key	Effective reading cursor verse for sequential reading.

reading_scope	Either "full_mushaf" or "single_surah" based on user preference.
reading_scope_surah	Specific surah ID when reading_scope is "single_surah".
ayahs_marked_today	Count from reading_daily_stats for today in ledger timezone.
at_reading_scope_end	Boolean: whether cursor has reached the end of the chosen scope.

Table 4: UserMe profile contract fields

3.6 Email/Password Login

The schemas.py module defines UserRegister and UserLogin Pydantic models for email/password authentication, however **no FastAPI routes** under the auth directory currently expose email/password sign-in endpoints. Users created via OAuth receive a random password hash, while the demo user uses the DEMO_USER_PASSWORD environment variable. The email/password schemas exist as a foundation for future implementation but are not wired to any active route handlers at this time.

4. Session ID Semantics

The axios client sends an **X-Session-ID** header populated from the localStorage key **asar_session_id**. This session identifier is critical for correlating chat messages, streak activities, and reading history within the application. The effective_session_id logic in common.py handles two distinct scenarios based on the authentication state of the request.

For **anonymous users** (no valid Bearer token), the system uses the client-provided session UUID directly. This allows unauthenticated users to maintain a consistent session across requests, preserving their chat history and streak data even before they create an account. For **authenticated users**, the system exclusively uses **user.asar_session_id**, creating and persisting one if it does not already exist. A stale X-Session-ID from another browser profile **cannot** attach chat, streak, or history data to the wrong account once a valid JWT is presented. The useAppSession hook overwrites localStorage with user.asar_session_id after the /auth/me call to ensure consistency.

5. Versioned REST API (/api/v1)

All API endpoints are versioned under **/api/v1** and documented through automatic OpenAPI specification available at the /docs endpoint. The following subsections detail each API domain with its available methods, authentication requirements, and functional descriptions. The Sanctuary chat UI calls POST /chat/message exclusively, while the legacy POST /chat path remains available for API clients and OpenAPI testing.

5.1 Chat

The chat API provides three endpoints serving different aspects of the AI-powered Quran companion experience. The verified chat endpoint uses strict anti-hallucination measures, the REST RAG endpoint provides multi-stage retrieval, and the insights endpoint generates personalized spiritual reflections based on session history.

Method	Path	Auth	Description
POST	/chat	Optional Bearer	Verified chat with anti-hallucination system prompt; persists messages; returns updated_streak_count from DB (computed, not incremented by chat alone).
POST	/chat/message	Optional Bearer	REST RAG chat with multi-turn history; returns answer + verses[]; streak unchanged.
GET	/chat/insights	Optional Bearer	AI-powered personalized spiritual reflections: 3 themes (rhythm, thematic echo, gentle prompts) based on session history.

Table 5: Chat API endpoints

5.2 Streak and Habit Ledger

The streak system implements a consecutive-day counting algorithm that tracks user engagement with daily Quran reading. The streak count is computed by taking the latest activity_date as an anchor and walking backward one calendar day at a time while each date exists in the streak_activities table. This represents consecutive ledger calendar days with at least one logged activity, not "days since first use." The POST /streak endpoint also handles optional reading cursor advancement and daily statistics updates for onboarded users, plus optional Quran Foundation activity synchronization when OAuth tokens are available.

Method	Path	Auth	Description
POST	/streak	Optional Bearer	Log mark-complete; upsert StreakActivity; optional reading cursor + daily stats; optional Foundation sync.
GET	/streak/{session_id}	Optional Bearer	StreakSnapshot with updated_streak_count.
GET	/streak/{session_id}/activities	Optional Bearer	Recent activities (newest first), query limit 1-500 (default 120).

Table 6: Streak and habit ledger endpoints

5.3 Quran Content (Proxied)

The Quran content endpoints proxy requests to the upstream Quran.com HTTP API, providing verse text in Uthmani script, translations in configurable languages, audio recitation URLs, and full surah metadata. These endpoints require no authentication, making them accessible to both anonymous and authenticated users. The implementation uses the shared `httpx.AsyncClient` for efficient connection pooling and supports optional OAuth client credentials for upstream API access.

Method	Path	Auth	Description
GET	/verse	None	Query by verse_key (surah:ayah); returns Uthmani, translation, optional audio_url.
GET	/translations	None	Translation resource catalog with optional language filter.
GET	/chapters	None	Full surah list (1-114) with metadata.
GET	/chapters/{chapter_id}	None	Single surah metadata (returns 404 if invalid).

Table 7: Quran content API endpoints

5.4 Chat History

The chat history endpoints provide session-scoped message persistence and retrieval. Both the verified chat (POST /chat) and REST RAG chat (POST /chat/message) append user and assistant messages to the same chat_messages table, meaning a session may contain turns from both pipelines if both were used. The delete endpoint removes all messages for a given session, providing a clean-slate mechanism for users who want to start fresh.

Method	Path	Auth	Description
GET	/history/{session_id}	Optional Bearer	Retrieve ordered chat messages for the effective session.
DELETE	/history/{session_id}	Optional Bearer	Delete all messages for the specified session.

Table 8: Chat history endpoints

5.5 Bookmarks (Authenticated Only)

The bookmarks API requires a valid Bearer token and provides full CRUD operations for verse bookmarks. Bookmarks support optional user notes and are designed for bi-directional synchronization with the Quran Foundation cloud service. Background tasks handle the push and delete operations against the remote API, ensuring the local response is fast while synchronization happens asynchronously. The sync status and remote bookmark ID are tracked for each local bookmark.

Method	Path	Auth	Description
GET	/bookmarks	Bearer	List user bookmarks with verse data and sync status.
POST	/bookmarks	Bearer	Create or update note; queues background push to Foundation.
DELETE	/bookmarks/{surah_id}/{ayah_number}	Bearer	Local delete; background remote delete if quran_bookmark_id set.

Table 9: Bookmarks API endpoints

6. Two Chat Pipelines

ASAR implements two distinct chat pipelines, both designed from the ground up to prevent hallucination and ground all responses in verified Quranic text. The Sanctuary chat UI uses the REST RAG pipeline exclusively (POST /chat/message), while the legacy Verified Assistant pipeline (POST /chat) remains available for API clients and OpenAPI documentation testing. Each pipeline follows a different approach to retrieval and prompt construction, offering different tradeoffs between speed and depth.

6.1 POST /chat - Verified Assistant

The Verified Assistant pipeline implements a strict, safety-first approach in the `generate_verified_chat_turn` function. The pipeline begins by short-circuiting greetings and meta-questions to a fixed welcome reply without fetching any verse data. For substantive questions, the system extracts an explicit `surah:ayah` reference from the text if present, or calls `search_verse_keys` against the Quran API to find relevant verses. If no matching keys are found, the system returns a fixed "cannot find a specific verse" response rather than fabricating content. When verses are located, the system fetches the Uthmani text, translation, tafsir excerpt, and optional audio URL. It then builds LLM messages using only the `VERIFIED_SYSTEM_PROMPT` and API-fetched facts, including up to 24 turns of recent DB history for multi-turn context. The LongCat completion uses low temperature and capped tokens to ensure focused, factual responses.

6.2 POST /chat/message - REST RAG

The REST RAG pipeline is a richer, multi-stage retrieval-augmented generation system implemented in `run_rest_rag_chat`. It begins by resolving the appropriate translation resource from `answer_language`, `translation_resource_id`, Urdu heuristics, and the configurable `QURAN_CHAT_TRANSLATION_RESOURCES_JSON` map. The retrieval stage proceeds through several optional but powerful phases: explicit key extraction, an optional query bridge (LLM call refining the user question into an English search phrase), an optional JSON query planner for multi-query decomposition, a Quran `/search` API call, a relevance gate trimming weak hits, and an optional neighbor context expansion adding surrounding ayahs. An optional alignment check provides a second LLM pass to verify answer alignment with retrieved verses. The pipeline returns a formatted answer plus structured verse cards with no streak side effects. All stages are controlled by environment-backed toggles (`quran_chat_relevance_*`, `quran_chat_query_bridge_*`, `quran_chat_query_planner_*`, `quran_chat_neighbor_*`, `quran_chat_alignment_*`) allowing fine-tuning without code changes.

7. Persistence (SQLite)

ASAR uses SQLite as its persistence layer, configured via the `DATABASE_URL` environment variable (default: `sqlite:///asar.db`). The schema is defined through SQLAlchemy ORM models in `domain.py` and covers six primary tables that support user accounts, session management, chat history, streak tracking, daily reading statistics, and bookmark synchronization. The stateless API tier

assumption means SQLite is suitable for single-instance deployments, while scale-out scenarios would require migrating to a shared database such as PostgreSQL.

Table	Purpose
users	Accounts: email, password hash, asar_session_id, OAuth tokens, onboarding state, reading cursor fields, optional ledger_timezone.
user_sessions	UUID session_id primary key; parent entity for messages and streak activity rows.
chat_messages	Per-session chat log storing both user and assistant messages with role differentiation.
streak_activities	Unique per (session_id, activity_date); stores ayah_read verse key for each ledger day.
reading_daily_stats	Per-user per-ledger-day marks_count powering the dashboard Flow Ring and analytics.
verse_bookmarks	Per-user verse bookmarks with quran_sync_status and remote ID fields for Foundation sync.

Table 10: Database schema tables and purposes

8. Configuration Reference

All configuration values are loaded from environment variables via Pydantic Settings (env_file=".env"), following the 12-factor app methodology. Secrets must never be committed to the repository and should be stored in environment variables or a secret manager. The following subsections document all configurable parameters organized by category.

8.1 Core and Security

Variable	Role
DATABASE_URL	SQLAlchemy connection string (default: sqlite:///./asar.db).
CORS_ORIGINS	Comma-separated allowed browser origins; defaults to localhost dev ports if unset.
JWT_SECRET_KEY	JWT signing secret (must be 24+ random characters in production).
JWT_ALGORITHM	JWT signing algorithm (default: HS256).

ACCESS_TOKEN_EXPIRE_MINUTES	JWT token lifetime in minutes.
ENABLE_DEMO_LOGIN	Boolean gate enabling demo account access (set false in production).
DEMO_USER_EMAIL / DEMO_USER_PASSWORD	Demo account credentials for instant access mode.

Table 11: Core and security configuration variables

8.2 LongCat (LLM)

Variable	Role
LONGCAT_API_KEY	API key for the OpenAI-compatible LLM service.
LONGCAT_BASE_URL	Base URL for LongCat completions endpoint.
LONGCAT_MODEL	Model identifier for LLM requests.
LONGCAT_CHAT_MAX_TOKENS	Maximum token cap for chat completions.
LONGCAT_CHAT_TEMPERATURE	Sampling temperature controlling response creativity.

Table 12: LongCat LLM configuration variables

8.3 Quran.com Content API

Variable	Role
QURAN_API_BASE_URL	Upstream HTTP API base URL.
QURAN_API_KEY	API key for Quran.com content endpoints.
QURAN_TRANSLATION_RESOURCE_ID	Default translation resource for verse responses.
QURAN_CONTENT_LANGUAGE	Primary content language code.
QURAN_SECONDARY_CONTENT_LANGUAGE	Secondary content language code.
QURAN_SEARCH_LANGUAGE	Language for Quran search queries.
QURAN_URDU_TRANSLATION_RESOURCE_ID	Urdu resource for chat/message heuristics.

QURAN_DEFAULT_VERSE_KEY	Fallback default verse key.
QURAN_AI_MAX_VERSES	Maximum verses returned in AI chat responses.

Table 13: Quran.com Content API configuration variables

8.4 REST RAG Tuning

The REST RAG pipeline supports extensive fine-tuning through a set of environment-backed configuration variables prefixed with `quran_chat_`. These include: relevance gate thresholds (`quran_chat_relevance_min_score`), query bridge settings (`quran_chat_query_bridge_enabled`, `quran_chat_query_bridge_model`), JSON query planner controls (`quran_chat_query_planner_enabled`, `quran_chat_query_planner_max_queries`), neighbor context expansion (`quran_chat_neighbor_enabled`, `quran_chat_neighbor_count`), alignment check options (`quran_chat_alignment_enabled`, `quran_chat_alignment_model`), and the translation resource JSON map (`QURAN_CHAT_TRANSLATION_RESOURCES_JSON`). All of these can be adjusted without code changes by modifying environment variables.

8.5 Quran Foundation OAuth and APIs

Variable	Role
QURAN_CLIENT_ID	OAuth client identifier for Quran Foundation user authentication.
QURAN_CLIENT_SECRET	OAuth client secret for Quran Foundation user authentication.
QURAN_OAUTH_AUTHORIZE_URL	OAuth authorization endpoint URL.
QURAN_OAUTH_TOKEN_URL	OAuth token exchange endpoint URL.
QURAN_OAUTH_REDIRECT_URI	Must be byte-for-byte identical to Foundation console registration.
QURAN_OAUTH_AUTHORIZE_SCOPES	Scopes requested during OAuth authorization flow.
QURAN_USER_API_BASE_URL	User API base URL for activity and bookmark endpoints.
QURAN_BOOKMARKS_URL	Override URL for bookmark synchronization endpoint.

FRONTEND_AFTER_OAUTH_URL	SPA base URL for post-login redirect with #asar_token fragment.
--------------------------	---

Table 14: Quran Foundation OAuth and API configuration variables

8.6 Ledger

Variable	Role
ASAR_LEDGER_TIMEZONE	Default IANA timezone for streak day boundaries when user has no override.

Table 15: Ledger configuration variable

8.7 Front-end Environment

Variable	Role
VITE_API_BASE_URL	API base path (e.g. /api/v1 or full engine URL).
VITE_ASAR_ENGINE_ORIGIN	Engine origin for OAuth start when SPA and API hosts differ.

Table 16: Front-end environment variables

9. Front-End: Routing and Guards

The frontend routing is defined in App.tsx using React Router with a layered guard composition. Public routes (/welcome, /welcome/oauth) require no authentication, while all other routes are wrapped in RequireAppAccess which enforces JWT validation. The OnboardingGate checks whether the user has completed the onboarding flow and redirects to /onboarding if not. AppShell provides the main navigation layout and wraps all primary application routes. This layered approach ensures security while providing a smooth user journey.

Route / Group	Guard / Notes
/welcome	Public; no guard. Entry point with demo login and Quran OAuth buttons.
/welcome/oauth	Public; no guard. OAuth fragment handler (QuranOAuthLanding.tsx).
/login, /register	Public; redirect to /welcome. Legacy route aliases.

RequireAppAccess wrapper	JWT required. Protects all routes below this boundary.
/onboarding	Inside RequireAppAccess. Onboarding flow without main shell.
OnboardingGate	If !user.onboarding_completed, redirects to /onboarding.
AppShell: /	Dashboard. Daily ayah, mark complete, streak constellation, flow ring.
AppShell: /dhikr	Dhikr counter UI. Local persistence only.
AppShell: /community	Community shell destination. Presentational content.
AppShell: /library	Bookmark library. GET/POST/DELETE /bookmarks when signed in.
AppShell: /support	Support hub with email and Discord links.
AppShell: /quran	Surah list page. GET /chapters.
AppShell: /quran/:surahId	Quran Reader page. GET /chapters/{id}, GET /verse.
AppShell: /focus	Focus mode. Distraction-free ayah view.
AppShell: /habits	Habit ledger heatmap. GET /streak/.../activities.
AppShell: /chat	AI Companion chat. GET/POST/DELETE history + POST /chat/message.
AppShell: /insights	AI Insight Results. GET /chat/insights.
AppShell: /settings, /settings/reading	User settings and reading preferences.
AppShell: /config	Configuration page. Local config patterns.
/mentor	Redirects to /chat.
*	Catch-all redirects to /.

Table 17: Front-end routing and guard composition

10. Front-End: Pages and Data Sources

Each front-end page consumes specific backend endpoints and local storage mechanisms to render its content. The following table maps every application route to its primary purpose and the backend endpoints or storage mechanisms it depends on. This provides a complete picture of the data flow from the backend through the API client to the user interface.

Route	Primary Purpose	Backend / Storage
/ (Dashboard)	Daily ayah, mark complete, streak, flow ring, translation selector	POST /streak, POST /auth/me/reading-restart, /auth/me fields via context, verse/streak via MoodAyahContext + engineDataCache
/dhikr	Dhikr counter UI	Local persistence only (dhikrStorage.ts)
/quran	Surah list	GET /chapters (QuranListPage.tsx)
/quran/:surahId	Quran Reader	GET /chapters/{id}, GET /verse via fetchVerseBundleDeduped (ReaderPage.tsx)
/library	Bookmark library	GET/POST/DELETE /bookmarks when signed in; local merge (bookmarksApi.ts)
/focus	Focus mode on current ayah	fetchVerseBundleDeduped (FocusPage.tsx)
/chat	AI Quran companion	GET /history, POST /chat/message, DELETE /history (ChatPage.tsx); prefs in chatPreferences.ts
/insights	AI Insight Results	GET /chat/insights (AI-driven reflections, Bento Grid layout) (InsightsPage.tsx)
/habits	Habit ledger heatmap	GET /streak/.../activities via cache helpers (HabitsPage.tsx)
/onboarding, /settings/reading	First-time or reading prefs	GET /chapters, PATCH /auth/me/onboarding (OnboardingPage.tsx)
/community, /support, /config	Shell destinations	No apiClient usage (presentational / local config patterns)

Table 18: Front-end pages and their data sources

11. Testing

The ASAR Sanctuary codebase employs a multi-layered testing strategy covering backend unit and integration tests, frontend unit tests, and end-to-end browser tests. The FastAPI engine exposes interactive OpenAPI documentation at /docs when running, providing an additional manual testing surface for exploring API behavior. The following table summarizes the testing stacks and their locations within the repository.

Area	Stack	Location
------	-------	----------

Backend (unit/integration)	pytest, pytest-asyncio, httpx, anyio	Back-End/tests/ (e.g. test_chat_rag_rest.py, test_quran_service.py)
Frontend (unit)	Vitest	Front-End/package.json script "test"
Frontend (e2e)	Playwright	Front-End/e2e/ (browser-based workflow tests)

Table 19: Testing stacks and locations

12. Operational Notes (12-Factor)

The ASAR Engine follows 12-factor app principles for configuration management, logging, and deployment. All configuration is externalized through environment variables, logs use structured formatting, and the API tier assumes statelessness with SQLite providing single-instance persistence. The following checklist and guidelines should be reviewed before any production deployment.

12.1 Production Checklist

Item	Notes
JWT_SECRET_KEY	Long random value (24+ characters); must differ from the repository default.
CORS_ORIGINS	Comma-separated SPA origins that call the API with credentials.
ENABLE_DEMO_LOGIN	Set to false if demo JWT must not exist in production.
DATABASE_URL	SQLite path or managed database URL for multi-instance deploys.
FRONTEND_AFTER_OAUTH_URL	Must match where the SPA handles #asar_token after Quran OAuth.
GET /health	Liveness endpoint returning {"status":"ok"}; use for probes and uptime checks.
Secrets management	LongCat, Quran Foundation, and demo Quran tokens belong only in environment or secret store.

Table 20: Production deployment checklist

12.2 OAuth Redirect URI Troubleshooting

When the Quran Foundation OAuth server (Hydra) reports a redirect_uri mismatch, follow this ordered checklist to resolve the issue. First, a new Client ID requires re-registering redirect URIs since the Foundation does not copy redirect

URLs when issuing new credentials. Second, the `QURAN_OAUTH_REDIRECT_URI` must be byte-for-byte identical to one allowed redirect URI, including scheme, host, port, and path. Third, the redirect URI must target the engine (e.g. `http://localhost:8000/callback`), not the Vite dev server. Fourth, the `GET /api/v1/auth/quran/redirect-uri-hint` endpoint returns the configured `redirect_uri` for comparison. Fifth, server logs from both the OAuth start and the engine startup record the configured `redirect_uri` for debugging. Additionally, `VITE_ASAR_ENGINE_ORIGIN` on the front-end must use the same host and port as `QURAN_OAUTH_REDIRECT_URI` so the PKCE cookie is sent on callback. The Vite proxy forwards `/api` to the engine without stripping the `/api` prefix.

12.3 General Operational Guidelines

Configuration should always use environment variables over editing code, and no secrets should ever be committed to the repository. Structured logging is configured in `logging_config.py` and imported from `main.py`. The API tier assumes statelessness, with SQLite providing single-file persistence suitable for single-instance deployments. For development, run the engine on port 8000 and the Vite dev server on port 5173, using the proxy to avoid CORS friction. The engine creates shared `httpx` and `AsyncOpenAI` clients during its async lifespan for efficient resource management. Scale-out scenarios would require migrating from SQLite to a shared database such as PostgreSQL.